

T7 / T11 / T15 深度技术指南：数据处理与方法链全解析

适用对象：北京航空航天大学人工智能学院 2026 年春季学期数理统计课程 分析选题：T7 幽灵舰队、T11 红海”生存博弈”、T15 夜间灯光”暗下去”

一、三题概览与快速对比

你选择的三个选题均属于遥感数据分析模块，但分别聚焦于海事异常检测（T7）、生存分析与因果推断（T11）、遥感面板数据与合成控制（T15）三个截然不同的技术方向。这三个选题共享”免费数据 + 现实冲突场景 + 可发表方法论”的共同优势，但在核心技术栈、编程门槛和统计理论深度上存在显著差异。以下对比表可帮助你快速建立整体认知：

对比维度	T7 幽灵舰队	T11 红海生存博弈	T15 夜间灯光
核心问题	如何从 AIS 数据识别制裁规避行为	AIS 开关是否降低遇袭风险	如何从夜光数据推断冲突损害
难度等级			
主要数据源	AISHub + MarineTraffic	AISHub + ACLED + IMO GISIS	NASA Black Marble + ACLED
核心统计方法	变点检测·Logistic·Capture-Recapture	生存分析·Cox·PSM·Rosenbaum 检验	面板回归·SCM·空间 Durbin 模型
编程依赖	Python (pandas, geopandas, ruptures)	Python (lifelines, statsmodels)	Python (geemap) + GEE JavaScript
数据预处理量	大（全球 AIS 日记录数千万条）	中（需匹配三套时空数据）	小（GEE 云端处理，免下载）
理论深度	中（变点检测 + 分类模型）	高（生存分析 + 因果推断双层结构）	中（面板数据 + SCM）
方法创新性	Capture-Recapture 用于海事领域较新颖	生存分析 + PSM 引入海事安全极少见	SCM+ 面板回归在夜光文献中较少
发表潜力	中-高	高	中-高
预计完成周期	4-6 周	5-7 周	4-5 周

从上述对比可见，T11 在统计理论深度和方法创新性上最为突出，适合追求学术深度的学生；T15 在数据获取便利性上独占优势（GEE 云端处理免去了大量数据下载工作），适合时间有限但希望产出高质量空间可视化成果的学生；T7 则在异常检测和估计理论方面提供了扎实的技术训练，且与海事政策直接挂钩，现实影响力强。

二、T7：幽灵舰队——AIS 异常行为与制裁规避识别

2.1 问题背景与研究价值

俄乌冲突爆发后，西方对俄罗斯实施了多轮石油禁运和航运制裁。为规避制裁，一支被称为”幽灵舰队”（Dark Fleet）的庞大船队应运而生——据新加坡国立大学 CIL 2025 年报告估计，该舰队规模已超过 **600 艘油轮**。这些船只通过三种主要手段隐匿行踪：关闭 AIS 发射器（产生信号中断 Gap）、船对船转运（Ship-to-Ship, STS）、以及伪造船旗和船舶身份。传统的海事监管依赖 AIS 数据，而幽灵舰队的存在直接挑战了这一体系的完整性。[\(Centre for International Law\)](#)

该选题的统计核心在于：如何从海量 AIS 数据中自动识别这些异常行为，并统计估计幽灵舰队的真实规模？这不仅是一个技术问题，更直接关联到制裁政策的有效性评估——如果无法准确估计规避规模，就无法判断制裁是否真正起到了预期效果。

2.2 数据获取与处理全流程

2.2.1 数据源的层次化结构 T7 需要整合三套异构数据源，每套数据在空间、时间和语义粒度上均有差异：

数据层	数据源	获取方式	关键字段	数据规模
AIS 动态层	AISHub / MarineCadastrtre.gov	免费 API / 直接下载	MMSI, BaseDateTime, LAT, LON, SOG, COG, VesselType	日记录数千万条
船舶静态层	MarineTraffic / IMO GISIS	API / 批量下载	MMSI, IMO, 船名, 船龄, 吨位, 船旗, 船东	数十万条
制裁清单层	OFAC / EU Sanctions List	公开网页	船名, IMO, 关联实体	数百条

AIS 数据是整个过程的基础。以美国 NOAA 提供的 AIS 数据为例，其标准 CSV 格式包含以下核心字段：MMSI（船舶唯一标识，9 位数字）、BaseDateTime（UTC 时间戳）、LAT/LON（WGS84 坐标）、SOG（对地速度，节）、COG（对地航向，度）、VesselType（AIS 船舶类型编码，如 70= 货船，80= 油轮）。Python 中可通过 `pandas.read_csv()` 直接读取，并用 `geopandas.GeoDataFrame` 转换为空间数据框以便后续地理操作。[\(Fused\)](#)

2.2.2 数据清洗的关键步骤 AIS 原始数据存在四类典型质量问题，必须在分析前逐一处理：

第一，坐标异常值。AIS 设备故障或人为干扰会导致坐标跳变——例如一艘船在 1 秒内从波斯湾”瞬移”到北冰洋。处理策略是基于速度阈值过滤：若相邻两条记录间的推算速度超过船舶物理上限（油轮通常不超过 25 节），则将后续记录标记为可疑。数学表达为：若 $d_{ij}/\Delta t > v_{\max}$ ，则记录 j 为异常，其中 d_{ij} 为 Haversine 距离， Δt 为时间间隔， $v_{\max}$ 取 30 节作为保守阈值。[\(ORBi\)](#)

第二，MMSI 冲突与伪装。幽灵舰队常使用伪造或盗用的 MMSI 编码。验证策略是将 AIS 中的 MMSI 与 IMO GISIS 注册数据库交叉比对——若某 MMSI 对应的船舶类型、吨位在两条记录间发生不合理跳变（如从油轮变为渔船），则标记为身份伪装嫌疑。

第三，重复记录去噪。AIS 接收网络存在重叠覆盖，同一船舶的同一时刻可能被多个基站记录。去重策略是按 MMSI + BaseDateTime 组合键分组，保留 SOG/COG 变化最平滑的一条记录（即与前后时刻运动学一致性最高的记录）。[\(ORBi\)](#)

第四，坐标系统一。不同数据源的坐标系可能存在微小差异（如 WGS84 vs CGCS2000），需通过 `pyproj` 库统一转换至 EPSG:4326 标准。对于大尺度分析（如全球航线），这种差异可忽略；但对于港口尺度的高精度分析，坐标系统一不可忽视。

2.2.3 特征工程：从原始轨迹到判别特征 特征工程是 T7 的核心技术环节——需要将原始 AIS 轨迹转化为可用于异常检测的统计特征向量。以下是六类关键特征及其计算方法：

特征类别	具体指标	计算方法	异常判别逻辑
信号连续性	AIS Gap 时长/频次	相邻记录时间差 > 阈值 (如 1 小时) 即计为 Gap	Gap 频繁且集中在制裁相关海域 → 高度嫌疑
空间行为	航线复杂度 (转弯频率)	单位距离内航向变化 >45° 的次数	异常复杂航线 (规避某区域) 或异常简单航线 (直穿禁区)
STS 行为	船对船靠近事件次数	两船距离 <500m 且持续 >30 分钟	STS 频繁 → 可能进行非法转运
船舶属性	船龄、船旗、船东	与制裁清单交叉比对	老旧船 + 便利旗 + 匿名船东 → 高风险组合

Table 3 – continued

特征类别	具体指标	计算方法	异常判别逻辑
时间模式	夜间航行比例	本地时间 20:00-06:00 的航行里程占比	刻意夜间活动 → 规避监测
港口关联	停靠俄罗斯港口频次	地理围栏判断 (港口缓冲区 10km)	频繁停靠受制裁港口 → 直接证据

特征提取的 Python 实现可借助 `movingpandas` 库进行轨迹分段和停留点检测，配合 `geopandas` 的空间操作（如缓冲区分析 `buffer()` 和空间连接 `sjoin()`）。(ORBi)

2.3 统计方法链详解

2.3.1 变点检测: CUSUM 与 PELT 算法 AIS 信号中断 (Gap) 是幽灵舰队最核心的暴露特征。但合法船舶也会因设备故障、进入无覆盖海域或靠近冲突区域时主动关闭 AIS 而产生 Gap。因此，区分”合法 Gap”和”规避性 Gap”是方法链的第一道关卡。

CUSUM (累积和) 算法 是经典的序列变点检测方法。其原理是累积观测值与参考均值的偏差，当累积量超过阈值时触发变点警报。对于 AIS Gap 检测，设 $g_t = \max(0, g_{t-1} + x_t - \mu - k)$ ，其中 x_t 为 t 时刻的 Gap 指示变量 (1=Gap, 0= 正常)， μ 为历史基线 Gap 率， k 为漂移参数。当 $g_t > h$ (阈值) 时，判定为异常 Gap 模式开始。(statisticsassignmenthelp.com)

PELT (Pruned Exact Linear Time) 算法 是 `ruptures` 库提供的高效变点检测方法，通过动态规划在 $O(n)$ 时间内找到最优变点分割。相比 CUSUM，PELT 的优势在于自动确定变点数量且支持多种损失函数 (如 L2 损失、L1 损失)。Python 实现极为简洁：

```
import ruptures as rpt

# gap_series: 0-1 时间序列, 1 表示 AIS Gap
algo = rpt.Pelt(model="L2", min_size=24).fit(gap_series)
change_points = algo.predict(pen=10) # pen 为惩罚参数, 控制变点数量
```

在实际应用中, 建议将 CUSUM 作为实时警报工具 (对每条船舶的 Gap 序列单独运行), PELT 作为事后批量分析工具 (对全球船队的 Gap 模式进行整体分段)。两种方法的结果可交叉验证，提高检测可靠性。(statisticsassignmenthelp.com)

2.3.2 Logistic 回归: 异常概率建模 提取多维特征后，需要构建一个可解释的判别模型来量化每艘船的制裁规避概率。Logistic 回归是首选工具——它不仅输出概率值，还能通过 odds ratio 解释各特征的风险贡献度。

模型设定为: $\log \frac{p_i}{1-p_i} = \beta_0 + \beta_1 \text{GapFreq}_i + \beta_2 \text{STSCount}_i + \beta_3 \text{FlagRisk}_i + \beta_4 \text{Age}_i + \beta_5 \text{RussiaPort}_i + \epsilon_i$ ，其中 p_i 为船舶 i 属于幽灵舰队的概率。

训练数据的标签构建是一个关键挑战——我们无法获得”真实的幽灵舰队名单”作为监督信号。替代策略是使用已公开的制裁船舶名单 (如 OFAC 清单) 作为正样本，经人工验证的合规船舶作为负样本。虽然这种标签存在噪声 (制裁名单可能不完整，合规船舶中可能混杂未发现的规避者)，但在大样本条件下，Logistic 回归对此类标签噪声具有较好的鲁棒性。(Centre for International Law)

模型评估采用 ROC 曲线和 AUC 指标。由于正负样本极不平衡 (幽灵舰队占总船队比例 <5%)，AUC 比准确率更适合作为评估标准。同时应关注精确率-召回率曲线 (PR 曲线)，因为在实际应用中，高精确率 (减少误报) 比高召回率 (减少漏报) 更具操作价值——海事执法资源有限，经不起大量误报消耗。

2.3.3 Capture-Recapture: 暗船数量估计 Logistic 回归能判断”这艘船是否可疑”，但无法回答”幽灵舰队总共
有多少艘”。**Capture-Recapture (标记-重捕)** 方法是估计未知总体规模的经典统计工具，源于生态学（估计野生动物
种群数量），在此被创新性地应用于海事领域。bangor.ac.uk

其核心思想是利用**两个独立观测源**的重叠信息推断总体规模。设：

- M = 第一来源（如 AIS 数据）检测到的可疑船舶数
- C = 第二来源（如 SAR 卫星影像）检测到的可疑船舶数
- R = 两个来源共同检测到的船舶数（重捕数）

则 Chapman 修正的 Lincoln-Petersen 估计量为： $\hat{N} = \frac{(M+1)(C+1)}{R+1} - 1$ ，方差估计为： $\widehat{Var}(\hat{N}) = \frac{(M+1)(C+1)(M-R)(C-R)}{(R+1)^2(R+2)}$ 。[\(NC State Repository\)](#)

在 T7 的应用场景中，两个”捕获源”可以是：

- **源 1：**基于 AIS Gap 检测的异常船舶集合
- **源 2：**基于 SAR 影像的暗船检测集合（关闭 AIS 但在 SAR 中可见的船舶）

这种方法的巧妙之处在于——即使每个来源都存在大量漏检（即灵敏度 <100%），只要两个来源的漏检机制相对独立，就
能通过重捕信息校正总体估计。[\(Aaltodoc\)](#)

2.4 技术栈与关键 Python 库

功能模块	推荐库	安装命令	核心用途
数据处理	pandas, numpy	pip install pandas numpy	CSV 读取、清洗、特征计算
空间分析	geopandas, shapely	pip install geopandas shapely	地理围栏、距离计算、空间连接
轨迹分析	movingpandas	pip install movingpandas	轨迹分段、停留点检测
变点检测	ruptures	pip install ruptures	CUSUM/PELT/Binary Segmentation
分类模型	scikit-learn	pip install scikit-learn	Logistic 回归、ROC/AUC 评估
数量估计	scipy	pip install scipy	Capture-Recapture 置信区间

三、T11：红海”生存博弈”——AIS 开关与遇袭风险因果推断

3.1 问题背景与研究价值

2023 年 11 月以来，也门胡塞武装在红海和亚丁湾发动了**近百次袭击**，目标包括商船、油轮和集装箱船。在这一背景下，一个引人注目的统计现象浮现：在所有被袭击的船舶中，开启 AIS 和关闭 AIS 的各占约一半——但**关闭 AIS 的船舶被击中的概率显著更低**。这一发现引发了核心因果问题：AIS 开关状态是否对遇袭风险存在因果效应？[\(Nature\)](#)

然而，简单比较两组的遇袭频率会陷入**自选择偏误**（Self-selection Bias）的陷阱——选择关闭 AIS 的船长可能本身就更为谨慎（如选择更远离也门海岸的航线、航速更快、吨位更小），这些因素本身就会降低遇袭风险。因此，需要引入**因果推断方法**来分离”AIS 开关”的纯处理效应。

该选题的学术价值极高——“关闭 AIS 减少被击中可能”目前仅有粗浅的频率统计支撑，**将生存分析与因果推断系统引入海事安全领域**在文献中极为罕见，具有明确的跨学科发表潜力。

3.2 数据获取与处理全流程

3.2.1 三套数据的时空匹配 T11 需要同时处理三套异构数据，并在时空维度上精确匹配——这是整个分析中**技术难度最高**的数据预处理环节。

数据层	数据源	时间粒度	空间粒度	关键字段
遇袭事件	ACLED	日度	点坐标 (经纬度)	event_date, latitude, longitude, event_type, fatalities
AIS 状态	AISHub	分钟级	点轨迹	MMSI, BaseDateTime, LAT, LON, Status(0= 航行,1= 锚泊, 等)
船舶属性	IMO GISIS	静态	无	MMSI, IMO, vessel_type, gross_tonnage, flag, year_built

时空匹配的核心逻辑是：对于每一起 ACLED 记录的遇袭事件，在 AIS 数据中找到**事发时刻前后特定时间窗口内、事发位置特定距离范围内的所有船舶**，并标记其中哪些船舶的 AIS 处于开启状态、哪些处于关闭状态。

具体实现上，可采用 `scipy.spatial.cKDTree` 构建 AIS 记录的空间索引，对每起遇袭事件进行**最近邻查询**。时间匹配则通过 `pandas.merge_asof()` 实现——该函数支持按最近时间戳合并，非常适合处理时间不同步的多源数据。匹配后，每起遇袭事件将对应一组”候选船舶”，每个候选船舶附带其 AIS 开关状态、距事发点距离、事发时刻前后的运动参数等变量。[\(arXiv.org\)](#)

3.2.2 生存数据集的构建 生存分析需要三类核心变量，需从匹配后的数据中精心构造：

持续时间 (Duration, T)：对于每艘通过红海/亚丁湾的船舶，定义其”风险暴露时间窗口”——从首次进入高风险区域（如曼德海峡 20 海里缓冲区）到离开该区域的时间长度。对于遇袭船舶，T = 进入至遇袭的时间；对于未遇袭船舶，T = 进入至离开的时间（**右删失数据**）。[\(TutorialsPoint\)](#)

事件指示 (Event, E)：二元变量，1= 在暴露窗口内遇袭，0= 未遇袭（删失）。

协变量 (Covariates, X)：包括 AIS 开关状态（核心处理变量）、船舶类型（油轮/集装箱/散货）、吨位、船旗（是否便利旗）、航速、距也门海岸距离、时段（白天/夜间）等。这些协变量在 Cox 模型中用于控制混杂效应，在 PSM 中用于匹配。

3.3 统计方法链详解

3.3.1 Kaplan-Meier 生存曲线与 Log-Rank 检验 **Kaplan-Meier 估计器**是非参数生存分析的基础工具，用于估计”生存函数” $S(t)$ ——即船舶在时刻 t 仍未遇袭的概率。其计算基于**乘积极限公式**： $\hat{S}(t) = \prod_{t_i \leq t} (1 - \frac{d_i}{n_i})$ ，其中 n_i 为 t_i 时刻处于风险集的船舶数， d_i 为 t_i 时刻遇袭数。[\(Plotiviy\)](#)

Python 实现使用 `lifelines` 库的 `KaplanMeierFitter`：

```
from lifelines import KaplanMeierFitter

kmf_on = KaplanMeierFitter()
```



```

kmf_on.fit(durations=ais_on['T'], event_observed=ais_on['E'], label='AIS ON')
kmf_off = KaplanMeierFitter()
kmf_off.fit(durations=ais_off['T'], event_observed=ais_off['E'], label='AIS OFF')

# 绘制对比曲线
kmf_on.plot_survival_function()
kmf_off.plot_survival_function()

```

两条曲线的视觉分离程度提供了初步证据——若 AIS 关闭组的生存曲线系统性地高于开启组，则暗示关闭 AIS 可能降低遇袭风险。**Log-Rank 检验**则提供严格的统计显著性评估： $H_0 : S_{\text{on}}(t) = S_{\text{off}}(t)$ vs $H_1 : S_{\text{on}}(t) \neq S_{\text{off}}(t)$ 。lifelines.statistics.logrank_test() 可直接输出卡方统计量和 p 值。(Plotivty)

3.3.2 Cox 比例风险模型：量化风险比 Kaplan-Meier 和 Log-Rank 仅比较两组的整体生存差异，无法控制混杂变量的影响。**Cox 比例风险模型**通过引入协变量解决了这一问题，是生存分析的”行业标准”。(towardsdatascience.com)

模型设定为： $h(t|X_i) = h_0(t) \exp(\beta_1 \text{AIS}_i + \beta_2 \text{Tonnage}_i + \beta_3 \text{Speed}_i + \beta_4 \text{Distance}_i + \dots)$ ，其中 $h_0(t)$ 为基线风险函数（非参数估计）， $\exp(\beta_1)$ 即为 AIS 关闭（vs 开启）的**风险比 (Hazard Ratio, HR)**。若 $\text{HR} < 1$ 且 $p < 0.05$ ，则表明在控制其他因素后，关闭 AIS 显著降低遇袭风险。(TutorialsPoint)

Python 实现极为简洁：

```

from lifelines import CoxPHFitter

cph = CoxPHFitter(penalizer=0.1)
cph.fit(df_survival, duration_col='T', event_col='E')
cph.print_summary() # 输出 HR、95%CI、p 值

```

比例风险假设检验是 Cox 模型的关键诊断——要求各协变量的风险比不随时间变化。lifelines 提供 proportional_hazard_test() 函数，若某变量违反假设，可考虑引入时变协变量或分层 Cox 模型。(lifelines)

3.3.3 倾向性得分匹配 (PSM)：因果效应纠偏 Cox 模型控制了观测到的混杂变量，但船长选择关闭 AIS 的决策可能还受未观测因素影响（如船长经验、船公司安全政策、情报信息等）。这些未观测混杂若同时影响 AIS 开关和遇袭风险，就会导致 Cox 估计的偏误。**倾向性得分匹配 (PSM)**通过构造”反事实对照组”来缓解这一问题。(Frontiers)

PSM 的核心步骤如下：

Step 1: 估计倾向性得分。以 AIS 开关状态为因变量 (1= 关闭, 0= 开启)，以所有观测协变量（船舶类型、吨位、航速、航线、时段等）为自变量，拟合 Logistic 回归 (或 Probit 模型)，得到每艘船的倾向性得分 $e(X_i) = P(\text{AIS}_i = 1|X_i)$ ——即该船”选择关闭 AIS”的预测概率。(MDPI)

Step 2: 匹配。对每艘关闭 AIS 的”处理组”船舶，在开启 AIS 的”对照组”中寻找倾向性得分最接近的匹配对象。常用算法包括**最近邻匹配** (Nearest Neighbor)、**卡尺匹配** (Caliper Matching, 限制最大得分差距) 和**核匹配** (Kernel Matching, 加权平均)。(Matheus Facure)

Step 3: 估计 ATT。在匹配后的平衡样本上，比较处理组和对照组的遇袭率差异： $\text{ATT} = E[Y(1) - Y(0)|D = 1]$ ，即”对于那些实际关闭了 AIS 的船舶，如果它们没有关闭 AIS，遇袭风险会如何变化”。(Frontiers)

Python 实现可使用 CausalML 库或手动实现：

```

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import NearestNeighbors

# Step 1: 估计倾向性得分
ps_model = LogisticRegression().fit(X_covariates, D_ais)

```

```
propensity_scores = ps_model.predict_proba(X_covariates)[: , 1]

# Step 2: 最近邻匹配 (1:1)
nn = NearestNeighbors(n_neighbors=1).fit(control_ps)
distances, indices = nn.kneighbors(treated_ps)

# Step 3: 计算 ATT
ATT = treated_outcomes.mean() - control_outcomes[indices.flatten()].mean()
```

3.3.4 Rosenbaum 敏感性检验：评估隐藏偏误 PSM 的可靠性依赖于条件独立假设（CIA）——即给定观测协变量后，AIS 开关决策与潜在结果独立。若存在未观测混杂（如船长经验），该假设即被违反。**Rosenbaum 边界敏感性分析**量化了”未观测混杂需要多强才能推翻结论”。([semanticscholar.org](https://www.semanticscholar.org))

其核心思想是：设 Γ 为处理组和对照组因未观测因素导致的选择概率比值上限。计算不同 Γ 值下 ATT 的置信区间，若 ATT 在 Γ 达到合理阈值（如 1.5 或 2.0）时仍保持显著，则结论对隐藏偏误具有稳健性。Python 中可通过 `sensitivity` 包或手动实现 Rosenbaum 边界计算。(naro.go.ug)

3.4 技术栈与关键 Python 库

功能模块	推荐库	安装命令	核心用途
生存分析	lifelines	<code>pip install lifelines</code>	KM 曲线、Log-Rank、Cox 模型
因果推断	CausalML	<code>pip install causalml</code>	PSM、IPTW、Uplift 建模
统计检验	scipy	<code>pip install scipy</code>	敏感性检验、置信区间
数据处理	pandas	<code>pip install pandas</code>	生存数据集构建
空间匹配	scipy.spatial	内置	cKDTree 最近邻查询

四、T15：夜间灯光”暗下去”——VIIRS 遥感冲突损害估计

4.1 问题背景与研究价值

人类经济活动产生的人工照明在卫星夜光影像上留下清晰印记。当冲突摧毁基础设施、迫使居民撤离时，相应区域的夜间灯光强度会显著下降。NASA 的 **VIIRS Day/Night Band (DNB)** 传感器以 **500 米分辨率、月度合成**的频率记录全球夜光数据，为冲突损害的大规模、低成本、高时效评估提供了独特数据源。([nih.gov](https://www.nih.gov))

2025 年发表的一项研究利用 VIIRS Nightfire 数据对乌克兰战争进行了系统分析，发现 **90 个定居点**的夜光模式在冲突后发生显著变化，其中 Hostomel、Bakhmut、Pokrovsk 等已知激战区的灯光衰减最为明显。基辅地区的灯光像素数量在冲突首月下降了 **65.2%**，平均强度下降了 **24.5%**。([nih.gov](https://www.nih.gov))

该选题的统计核心在于：**如何从夜光时间序列中分离”冲突损害信号”与”常规经济波动”**，并构建反事实情景进行因果推断？

4.2 数据获取与处理全流程

4.2.1 NASA Black Marble 数据：GEE 云端处理 T15 的最大技术优势在于**数据获取和预处理的极度便利性**。VIIRS DNB 数据已完整接入 Google Earth Engine (GEE) 平台，无需下载 TB 级原始影像，全部处理可在云端完成。GEE 中的 VIIRS DNB 数据集标识为 `NOAA/VIIRS/DNB/MONTHLY_V1/CMSS/LC_FG`，包含 `avg_rad` 波段（平均辐射亮度，单位 $nW/cm^2/sr$ ）。该数据集已对杂散光进行校正并过滤了低质量数据，**开箱即用**。([Github](https://github.com))

Python 通过 `geemap` 库调用 GEE 的 Python API:

```
import geemap
import ee

# 初始化 GEE
ee.Initialize()

# 加载 VIIRS DNB 月合成数据
viirs = ee.ImageCollection("NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG") \
    .filterDate('2020-01-01', '2024-12-31') \
    .select('avg_rad')

# 定义研究区（以乌克兰某州为例）
roi = ee.FeatureCollection("FAO/GAUL/2015/level1") \
    .filter(ee.Filter.eq('ADM1_NAME', 'Kyiv'))

# 计算区域月度平均夜光强度
ts = viirs.map(lambda img: img.reduceRegions(
    collection=roi,
    reducer=ee.Reducer.mean(),
    scale=500
)).map(lambda f: f.set('date', img.date().format('YYYY-MM'))))
```

对于大作业级别的分析，推荐以乌克兰的州（Oblast）或城市为分析单元，这样可以获得约 20-30 个空间单元的月度面板数据，足以支撑面板回归和 SCM 分析。worldbank.github.io

4.2.2 面板数据结构的构建 从 GEE 导出的数据需要整理为平衡面板格式（Balanced Panel），即每个空间单元（i）在每个时间点（t）都有观测值。标准的面板数据结构如下：

unit_id	date	avg_rad	conflict_intensity	distance_to_frontline	treatment
Kyiv	2022-01	45.2	0	0	0
Kyiv	2022-03	15.8	8	0	1
Lviv	2022-01	32.1	0	500	0
Lviv	2022-03	31.5	0	500	0

其中 `conflict_intensity` 可从 ACLED 数据聚合得到（如该月该州内的冲突事件数），`distance_to_frontline` 可利用 `liveuamap` 等前线数据计算，`treatment` 为处理状态指示变量（前线州 =1，后方州 =0）。nih.gov

数据去噪是预处理的关键环节。VIIRS DNB 存在两类噪声源：云层遮挡（导致该月该像素无有效观测）和极光/火灾/渔船（产生临时性的亮度异常）。处理策略包括：使用 GEE 的 QA 质量标志过滤低质量像素；对月度序列进行 3 个月滑动平均平滑；以及剔除已知的永久性热源（如工业区），这些可通过对比战前基线识别。[MDPI](https://mdpi.com)

4.3 统计方法链详解

4.3.1 面板固定效应回归：基准模型 面板固定效应（Fixed Effects, FE）模型是 T15 的基准分析工具，其优势在于通过组内变换消除不随时间变化的遗漏变量（如地理区位、产业结构、基础设施存量等）。模型设定为：

$$Y_{it} = \alpha_i + \lambda_t + \beta \cdot \text{Conflict}_{it} + \gamma \cdot X_{it} + \epsilon_{it}$$

其中 Y_{it} 为 i 州 t 月的夜光强度， Conflict 为核心解释变量（可采用二元处理变量或 ACLED 冲突事件数的连续度量）， α_i 为州固定效应， λ_t 为时间固定效应（控制全国层面的经济周期、季节波动等）。系数 β 即为”冲突对夜光的平均因果效应”。

Python 可使用 `linearmodels` 库的 `PanelOLS`：

```
from linearmodels.panel import PanelOLS

model = PanelOLS.from_formula('avg_rad ~ Conflict + EntityEffects + TimeEffects',
                              data=df_panel)

results = model.fit(cov_type='clustered', cluster_entity=True)
print(results.summary)
```

聚类标准误（clustered at entity level）是必须的——同一州不同月份的误差项存在自相关，若不聚类会严重低估标准误、夸大显著性。

4.3.2 合成控制法 (SCM)：单元因果推断 面板 FE 模型估计的是平均处理效应，但若关心特定高冲突州（如 Donetsk 或 Kharkiv）的个体处理效应，合成控制法（Synthetic Control Method, SCM）是更精细的工具。SCM 由 Abadie 等人于 2003 年提出，通过从”捐赠池”（未受冲突影响的州）中加权组合构造一个”合成控制州”，使其在冲突前的夜光轨迹与处理州尽可能匹配。[\(동아시아경제리뷰\)](#)

SCM 的优化目标为： $\min_W \sum_{t=1}^{T_0} (Y_{1t} - \sum_{j=2}^{J+1} w_j Y_{jt})^2$ ，约束条件为 $w_j \geq 0$ 且 $\sum w_j = 1$ 。求解后，合成控制州的冲突后轨迹即为”如果没有冲突”的反事实预测，与实际轨迹的差额即为处理效应。

Python 可使用 `pysynthdid` 或 R 的 `Synth` 包实现。SCM 的统计推断依赖置换检验（Placebo Test）——对捐赠池中的每个州逐一假装其为处理州，计算安慰剂效应的分布，若真实处理效应位于分布的极端尾部，则统计显著。[\(동아시아경제리뷰\)](#)

4.3.3 空间 Durbin 模型：溢出效应分析 冲突损害可能存在空间溢出——前线的炮火不仅影响直接交火区，还会通过难民涌入、供应链中断、投资信心下降等渠道波及邻近地区。空间 Durbin 模型（Spatial Durbin Model, SDM）通过引入空间滞后项捕捉这种溢出效应：

$$Y_{it} = \rho WY_{it} + \beta X_{it} + \theta W X_{it} + \alpha_i + \lambda_t + \epsilon_{it}$$

其中 W 为空间权重矩阵（通常基于州间地理距离或共享边界构建）， ρ 为被解释变量的空间自回归系数， θ 为解释变量的空间滞后系数。`pysal` 库的 `spreg` 模块提供了 SDM 的实现。[\(Nature\)](#)

Moran’s I 指数应在建模前计算，以验证夜光数据的空间自相关性——若 Moran’s I 显著为正，则空间模型比标准面板模型更适合。

4.4 技术栈与关键库

功能模块	推荐库/平台	获取方式	核心用途
遥感数据处理	Google Earth Engine	免费注册 (gee.google.com)	VIIRS 数据云端筛选、聚合、导出
GEE Python 接口	geemap	<code>pip install geemap</code>	Python 调用 GEE API
面板回归	linearmodels	<code>pip install linearmodels</code>	FE/RE/面板工具变量
合成控制	pysynthdid / Synth(R)	<code>pip install pysynthdid</code>	SCM 估计与置换检验

Table 8 – continued

功能模块	推荐库/平台	获取方式	核心用途
空间统计	pysal	<code>pip install pysal</code>	Moran' s I、空间 Durbin 模型
数据可视化	matplotlib, seaborn	<code>pip install matplotlib seaborn</code>	夜光时序图、空间分布图

五、三题系统对比与决策建议

5.1 方法链深度对比

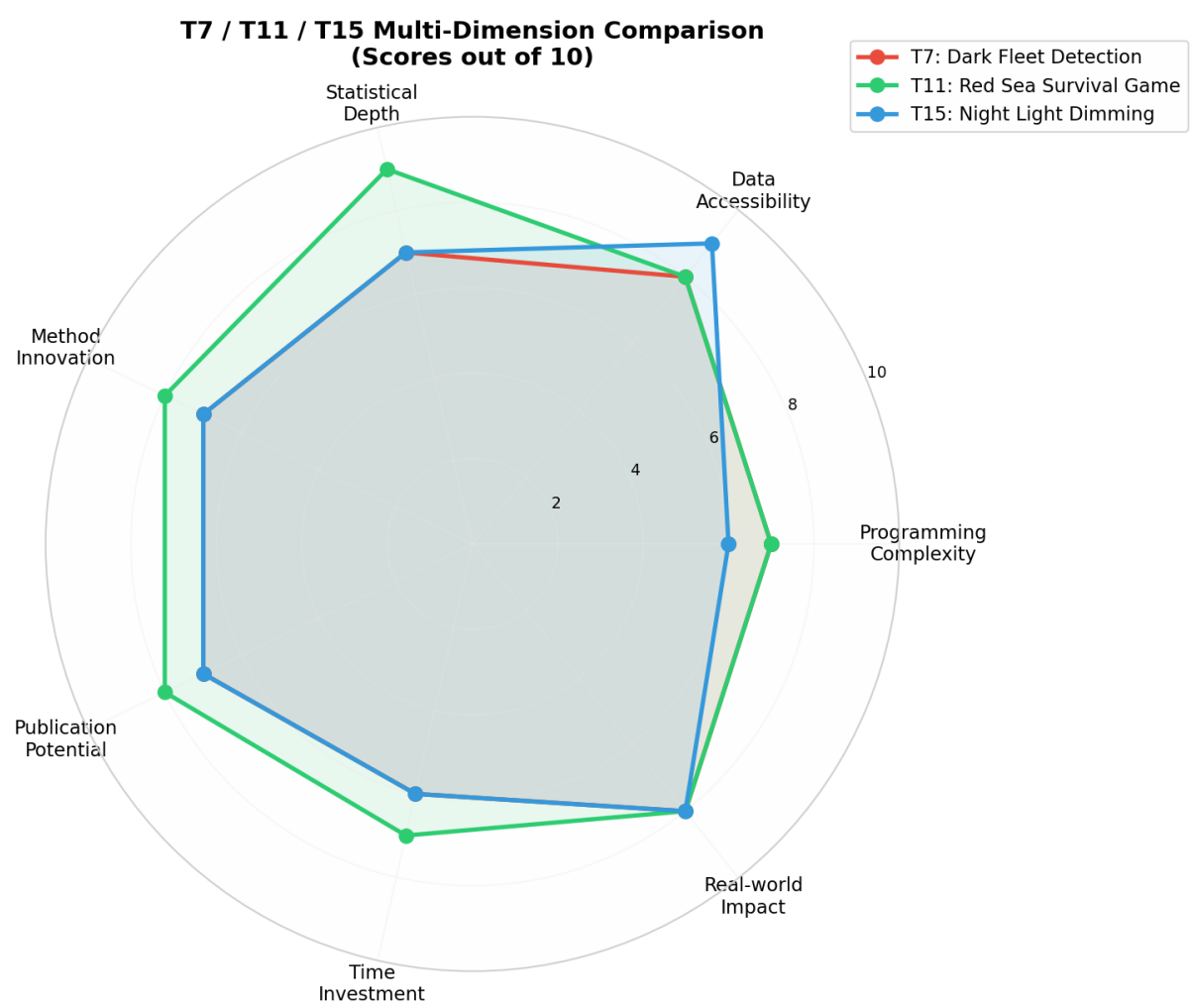


Figure 1: 三题多维度雷达对比

雷达图从七个维度量化了三题的差异。T11 在**统计深度** (9/10) 和**方法创新** (8/10) 上显著领先——生存分析与因果推断的双层结构在整个课程中独一无二；T15 在**数据可及性** (9/10) 上独占优势——GEE 云端处理彻底免去了数据下载和存储的烦恼；T7 在**现实影响** (8/10) 和**编程复杂度** (7/10) 上表现均衡——异常检测的技术栈与工业界数据科学岗位高度对口。

5.2 数据处理工作量对比

工作环节	T7	T11	T15
数据下载	大（AIS 日数据需批量下载）	中（三套数据分别获取）	极小（GEE 云端处理，免下载）
数据清洗	大（坐标去噪、MMSI 验证）	中（时空匹配复杂）	小（GEE 已做基础去噪）
特征工程	大（6 类特征需逐一计算）	小（生存变量直接定义）	小（面板变量直接聚合）
模型计算	中（Logistic+CR 估计）	大（KM→Cox→PSM→Rosenbaum）	中（GEE+SCM+SDM）
结果可视化	中（轨迹图 + ROC+CR 结果）	大（KM 曲线 + 森林图 + PSM 平衡图）	大（夜光时序 + 空间分布图）
总预估工时	40-60 小时	50-70 小时	35-50 小时

5.3 分场景推荐

场景一：“我想学习因果推断，追求方法论深度” 推荐：T11 红海”生存博弈”

T11 是整个课程中唯一系统整合生存分析与因果推断的选题。你将完整经历从 Kaplan-Meier 非参数估计 →Cox 半参数回归 →PSM 倾向性得分匹配 →Rosenbaum 敏感性检验的方法链，这一链条在计量经济学和生物统计学中具有极高的通用性。完成后，你不仅掌握了一套完整的因果推断工具箱，还能将其迁移到医学临床试验、金融风险管理、政策评估等无数应用场景。[\(TutorialsPoint\)](#)

关键挑战：时空数据匹配的技术实现、Cox 模型比例风险假设的诊断、PSM 匹配质量的平衡检验。

场景二：“我想快速产出高质量空间可视化成果” 推荐：T15 夜间灯光”暗下去”

T15 的 GEE 云端处理架构使其成为三题中”数据工作量最小、可视化冲击力最强”的选择。你无需下载任何遥感影像，仅需几十行 Python 代码即可提取整个乌克兰的夜光时间序列，配合 matplotlib 和 geemap 可生成令人印象深刻的时空变化图。合成控制法的反事实对比图（“如果 Donetsk 没有冲突，夜光会如何？”）具有极强的叙事力和政策说服力。[\(Github\)](#)

关键挑战：GEE 的 Python API 学习曲线、SCM 置换检验的计算耗时、空间权重矩阵的合理设定。

场景三：“我对海事数据感兴趣，想做有现实影响力的分析” 推荐：T7 幽灵舰队

T7 直接回应了当前国际制裁政策的核心痛点——如何量化评估制裁规避规模。你的分析结果可被直接引用到政策简报、学术研究或媒体报道中。Capture-Recapture 方法在生态学中极为成熟，但应用于海事领域仍属前沿，具有明确的方法论创新空间。此外，AIS 数据的处理经验（轨迹清洗、特征工程、空间分析）与工业界数据科学岗位的技能要求高度吻合。[\(Fused\)](#)

关键挑战：海量 AIS 数据的存储与计算效率、标签噪声对 Logistic 回归的影响、Capture-Recapture 假设的合理性验证。

5.4 技术准备清单

无论选择哪一题，建议在**第一周**内完成以下准备工作：

准备项	T7	T11	T15
注册 GEE 账号	否	否	是（必需）

Table 10 – continued

准备项	T7	T11	T15
安装 Python 库	pandas, geopandas, ruptures, sklearn	lifelines, causalmml, scipy	geemap, linearmodels, pysal
获取 API 密钥	AISHub API Key	ACLED 账号	GEE 认证
下载样本数据	1 天 AIS 数据 (测试解析)	ACLED 样本 (测试格式)	运行 GEE 示例代码
阅读对标论文	arXiv:2502.01503	Rosenbaum (2002)	Abadie et al. (2010)
撰写分析计划	明确研究区域和时间段	定义暴露窗口和事件	确定处理州和捐赠池

六、总结

T7、T11、T15 三个选题代表了遥感数据分析模块中三条各具特色的技术路线：T7 以异常检测和估计理论为核心，强调从海量轨迹数据中挖掘行为模式；T11 以生存分析和因果推断为核心，追求”观察数据中的因果答案”这一统计学终极命题；T15 以面板数据和合成控制为核心，利用遥感时序的反事实框架评估冲突影响。

三个选题均具备 100% 免费数据、明确的方法论对标、现实政策关联三大优势，差异主要体现在技术栈的偏好、编程工作量的分布和统计理论深度的梯度。选择时无需过分纠结”哪个更好”，而应思考”哪个更适合我当前的技术背景和未来的发展方向”。统计学的最高境界不是方法的堆砌，而是用最适合的工具回答最重要的问题——无论你选择哪一题，只要深入走完完整的方法链，都必将收获一段极具价值的学习体验。